NAOMI / BDC: Complete Step-by-Step Build Guide

Nov 9, 2026 · @Someone

Overview

NAOMI turns raw sales history into a recommended price and a plain-English explanation of why. Build the phases in order: each one consumes the output of the one before it.

The restaurant analogy. The forecast says how many customers will walk in. Elasticity says how many walk out if you raise menu prices. The optimizer picks the price that earns the most. The simulator asks "what if ingredients cost 10% more?". The LLM writes the manager's one-page brief.

How data flows

1. Data ingestion (Phases 2-3): raw M5 files become a clean table, then model-ready windows.
2. Forecasting (Phase 4): predicts units sold per SKU for the next N days.
3. Pricing (Phase 5): estimates how demand reacts to price and finds the profit-maximizing price.
4. Financial planner (Phase 5): converts units and price into revenue, profit and margin.
5. Scenario simulator (Phase 6): reruns steps 2-4 under shocks and reports the difference.
6. LLM narrative (Phase 8): turns the numbers into an executive summary.

The FastAPI backend (Phase 6) exposes all of this, and the dashboard (Phase 7) displays it.

How each step is written. Every step has four parts: **What** you do, **Why** it matters, **How** to do it, and **Check** to confirm it worked. Your status report marks Phases 1-4, 6 and 7 as done and Phases 5 and 8 as open, so the open phases get the deepest treatment, including the maths.

Key terms

Term	Meaning
SKU	One product (a stock-keeping unit); NAOMI uses 5

Term	Meaning
Window	The last N days fed to the LSTM as input (e.g. 28)
Horizon	How many days ahead you predict (e.g. 14)
Leakage	Information from the future sneaking into training, which inflates scores falsely
Elasticity (e)	% change in demand for a 1% change in price; negative for normal goods
MAE	Mean absolute error, in units sold
MAPE	Mean absolute percentage error; breaks when actual sales are 0

Phase 1: Project scaffolding and schemas

Goal: a repo and shared data contracts so five people can build in parallel without breaking each other. Output: `requirements.txt`, `src/config.py`, `src/data/schemas.py`.

Step 1.1: Create the repo structure

- **What:** one Git repo with folders `src/` (code), `tests/`, `docs/`, `presentation/`, plus a `main` branch and one feature branch per member (`feat/pricing-finance`, `feat/simulation-api`, `feat/dashboard-ui`, `feat/presentation-llm`).
- **Why:** separate branches let members work without overwriting each other. `main` always holds the verified, working version.
- **How:** `git init`, create folders, add `__init__.py` in each `src/` subfolder, add a `.gitignore` (ignore `__pycache__`, `.venv`, large raw data, `.env` for API keys), push to GitHub.
- **Check:** every member can clone, create a branch and push.

Step 1.2: Pin dependencies in `requirements.txt`

- **What:** list every library with a version.
- **Why:** "works on my machine" bugs usually come from different library versions.

- **How:** include pandas, numpy, scipy, statsmodels (OLS for elasticity), scikit-learn (Ridge baseline, scalers), torch (LSTM), fastapi, uvicorn, pydantic, httpx (API tests), plotly, dash, pytest. Create a virtual environment and run `pip install -r requirements.txt`.
- **Check:** a fresh virtual environment installs cleanly and `pytest --collect-only` runs.

Step 1.3: Write `src/config.py`

- **What:** one file holding every constant: file paths, the SKU list, window length, horizon, train/val/test fractions, random seed, assumed cost ratio, price-search band.
- **Why:** if the window length is hard-coded in five files, changing it means five edits and one missed one. One config file means one edit.
- **How:** use plain constants or a dataclass. Suggested defaults: `WINDOW = 28`, `HORIZON = 14`, `SEED = 42`, `COST_RATIO = 0.70`, `PRICE_BAND = 0.30`. Seed numpy and torch from `SEED` so runs are reproducible.
- **Check:** no other file contains a magic number such as 28 or 0.7.

Step 1.4: Define schemas in `src/data/schemas.py`

- **What:** typed Pydantic models for every object that crosses a module boundary: a data row, a forecast request and response, a pricing request and response, a scenario request and response.
- **Why:** this is the contract. Member 2 returns a `PricingResponse`, Member 3 consumes it, and Member 4 plots it. If everyone agrees on the fields first, nobody waits for anyone.
- **How:** for example `ForecastRequest(sku_id: str, horizon: int = 14)` and `ForecastResponse(sku_id, dates: list[date], mean: list[float], lower: list[float], upper: list[float])`. Add validators such as `horizon` between 1 and 28.
- **Check:** invalid input (negative horizon, unknown SKU) raises a validation error.

Step 1.5: Agree on the workflow

- **What:** a short `CONTRIBUTING.md`: branch naming, pull request rules, "tests must pass before merge".
- **Why:** it prevents merge chaos in Phase 9.
- **Check:** the first pull request is reviewed and merged using this process.

Phase 2: Dataset curation (Walmart M5)

Goal: turn three huge raw M5 files into one small, clean table of 5 products over 1,913 days. Output: `walmart_m5_curated.csv` (5 SKUs × 1,913 days = 9,565 rows).

Step 2.1: Download the raw data

- **What:** from the Kaggle "M5 Forecasting - Accuracy" competition, get `sales_train_evaluation.csv`, `calendar.csv` and `sell_prices.csv`.
- **Why:** M5 is real Walmart data with daily unit sales, prices and event calendars. It has the price variation you need for elasticity, which synthetic data would only fake.
- **How:** use the Kaggle website or `kaggle competitions download -c m5-forecasting-accuracy`. Keep raw files out of Git (they are large).
- **Check:** `sales_train_evaluation.csv` has columns `id, item_id, dept_id, cat_id, store_id, state_id, d_1 … d_1941`.

Step 2.2: Choose 5 SKUs deliberately

- **What:** pick 5 item-store combinations by rule, not at random.
- **Why:** elasticity is estimated from how sales moved when the price moved. A product whose price never changes gives you nothing to estimate. A product that sells 0 most days gives noisy data.
- **How:** for each candidate compute (a) mean daily sales, (b) share of zero-sales days, (c) number of distinct prices, (d) price range as % of mean price. Keep SKUs with healthy volume, under about 30% zero days, and at least 4-5 distinct prices. Prefer a mix of departments (e.g. foods, household) so the model is tested on different behaviours.
- **Check:** a small table of the 5 chosen SKUs with those four statistics, saved in `docs/`.

Step 2.3: Melt wide sales into long format

- **What:** the raw file has one column per day (`d_1`, `d_2`, …). Convert to one row per SKU per day.
- **Why:** models and joins need rows, not 1,913 columns.
- **How:** `pd.melt(df, id_vars=["item_id","store_id"], var_name="d", value_name="units")`. The result has 5 × 1,913 = 9,565 rows.
- **Check:** the row count is exactly 9,565 and each SKU has 1,913 rows.

Step 2.4: Merge the calendar

- **What:** join `calendar.csv` on the `d` column.
- **Why:** this adds the real date, weekday, month, event names and SNAP flags (US food-stamp days that boost demand), plus `wm_yr_wk`, the week code used to look up prices.
- **How:** `df.merge(calendar, on="d", how="left")`, then convert `date` to datetime.
- **Check:** no null dates, and dates are strictly increasing within each SKU.

Step 2.5: Merge weekly prices

- **What:** join `sell_prices.csv` on `store_id`, `item_id` and `wm_yr_wk`.
- **Why:** M5 prices are stored per week, not per day. The week code links each day to its price.
- **How:** a left merge keeps all sales rows. Rows with no price mean the item was not yet on sale that week.
- **Check:** count the null prices.

Step 2.6: Handle missing prices

- **What:** decide what to do with null prices.
- **Why:** a null price before launch is not a real zero price, and leaving nulls breaks the log in the elasticity model.
- **How:** drop rows before the first valid price, or back-fill from the first known price if the gap is small. Document which you chose. If you drop rows your total will fall below 9,565, so pick SKUs with full price history.
- **Check:** zero nulls in `price`.

Step 2.7: Save and sanity-check

- **What:** write `walmart_m5_curated.csv` and run validation checks.
- **Why:** one bad row here corrupts everything downstream.
- **How:** assert no nulls, no duplicate (SKU, date) pairs, no negative units, no zero or negative prices, and continuous daily dates per SKU. Plot units and price over time per SKU and look at them.
- **Check:** a short `tests/test_data.py` encoding these assertions passes.

Phase 3: Feature pipeline and windowing

Goal: turn the clean table into numeric inputs the LSTM can learn from, without leaking the future. Output: `src/data/pipeline.py`, `src/data/dataset.py`, and tests.

Analogy. Predicting tomorrow's sales is like a student sitting an exam. Features are the notes you allow them. Leakage is slipping the answer key into the notes: the score looks great, then collapses in the real exam.

Step 3.1: Build lag features

- **What:** for each SKU add columns holding sales 1, 7, 14 and 28 days ago.
- **Why:** yesterday's and last week's sales are the strongest predictors of today's. A lag of 7 captures "same weekday last week".
- **How:** `df.groupby("sku")["units"].shift(k)`. Group by SKU so one product's history never bleeds into another's.
- **Check:** the first `k` rows of each SKU are NaN for lag `k`, and nowhere else.

Step 3.2: Build rolling statistics

- **What:** rolling mean and standard deviation over 7 and 28 days.
- **Why:** these smooth daily noise and tell the model the recent level and volatility.
- **How:** `df.groupby("sku")["units"].shift(1).rolling(7).mean()`. **The `shift(1)` is critical:** without it, today's sales are inside today's average, which is leakage.
- **Check:** change today's sales to a huge number and confirm today's rolling feature does not change.

Step 3.3: Calendar features

- **What:** day of week, month, an event flag, SNAP flag, weekend flag.
- **Why:** retail demand has strong weekly and seasonal rhythms. Encode day of week as sin/cos (`sin(2π·dow/7)`) so Sunday and Monday are "close", not 6 apart.
- **Check:** values fall in the expected ranges.

Step 3.4: Price features

- **What:** the price, the price change vs yesterday, and relative price (price divided by the SKU's rolling average price).

- **Why:** this is how the forecaster sees promotions and the link to your elasticity model in Phase 5.
- **Check:** relative price is about 1.0 on average and below 1 on discount days.

Step 3.5: Split chronologically

- **What:** split each SKU's timeline into train (first ~70%), validation (next ~15%) and test (last ~15%).
- **Why:** in production you only ever predict the future from the past. A random shuffle would let the model train on days after the ones it is tested on, giving fake accuracy.
- **How:** cut by date index, never by `train_test_split(shuffle=True)`.
- **Check:** the max date of train is less than the min date of validation, which is less than the min date of test.

Step 3.6: Scale features without leakage

- **What:** standardize numeric features (subtract mean, divide by standard deviation).
- **Why:** LSTMs train poorly when features sit on very different scales (price 3.5 vs sales 400).
- **How:** fit the scaler on the **train split only**, then apply that same fitted scaler to validation and test. Fitting on all data lets test-set statistics leak into training. Save the scaler (joblib) so the API can reuse it.
- **Check:** train features have mean ≈ 0 and std ≈ 1; validation and test do not exactly.

Step 3.7: Drop NaN rows and create sliding windows

- **What:** build `(input window, target)` pairs.
- **Why:** the LSTM reads a sequence of past days and predicts the next ones.
- **How:** for window length `W = 28` and horizon `H = 14`, a sample at day `t` has input = features of days `t-W+1 … t` (shape `28 × n_features`) and target = units for days `t+1 … t+H`. Slide by one day. Implement as a PyTorch `Dataset` with `__len__` and `__getitem__`, and wrap it in a `DataLoader` (batch size 32-64; shuffle **training windows** is fine, because each window is self-contained and already past-only).
- **Check:** for a hand-picked sample, print the window's last date and the target's first date; the target starts exactly one day later.

Step 3.8: Write the tests

- Shapes are `(batch, 28, n_features)` and `(batch, 14)`.
- No NaN or infinite values.
- No overlap between train, validation and test dates.
- Window-target alignment (above).
- Scaler statistics come from train only.

Phase 4: Demand forecasting engine

Goal: predict units sold for the next 14 days per SKU, and prove the LSTM is worth using by beating simple baselines. Output: `src/models/` code, `pytorch_lstm.pt`, `docs/ablation_benchmark_results.csv`.

Step 4.1: Build four baselines first

- **What:** Naive (tomorrow = today), Seasonal Naive (tomorrow = same weekday last week), Moving Average (mean of last 7 or 28 days), and Ridge regression on the engineered features.
- **Why:** a baseline sets the bar. If a complex LSTM cannot beat "same as last week", it is not adding value. Retail data is so seasonal that Seasonal Naive is often hard to beat.
- **How:** each baseline exposes the same interface, `fit(train)` and `predict(history, horizon)`, so the benchmark loop treats them identically. Ridge adds an L2 penalty that shrinks coefficients and stops overfitting to correlated lags.
- **Check:** each baseline returns an array of shape `(horizon,)` and runs in seconds.

Step 4.2: Design the LSTM

- **What:** a 2-layer stacked PyTorch LSTM followed by a linear head that outputs 14 values.
- **Why:** an LSTM keeps a memory cell that carries information across the 28-day window, so it can learn weekly patterns plus promotion effects.
- **How:** `nn.LSTM(input_size=n_features, hidden_size=64, num_layers=2, dropout=0.2, batch_first=True)`. Take the last time step's hidden output, pass it through `nn.Linear(64, 14)`. Start with hidden size 64; try 128 only if it underfits.
- **Check:** a dummy batch of shape `(32, 28, n_features)` returns `(32, 14)`.

Step 4.3: Choose the loss, optimizer and early stopping

- **What:** Huber loss, Adam optimizer (learning rate about 1e-3), and early stopping on validation loss.
- **Why:** retail sales have spikes (holidays). Squared error lets one spike dominate training. Huber behaves like squared error for small errors and like absolute error for large ones, so it is robust to spikes. Early stopping halts when validation loss stops improving, which prevents memorising the training set.
- **How:** the Huber loss with threshold δ is:

```
L_\delta(a) = \begin{cases} \tfrac{1}{2}a^2 & |a| \le \delta \\ \delta\,(|a|
- \tfrac{1}{2}\delta) & |a| > \delta \end{cases}
```

where `a = actual − predicted`. Train for up to ~100 epochs, stop after 10 epochs with no validation improvement (patience = 10), and keep the weights from the best epoch.

- **Check:** both training and validation loss fall at first; if validation rises while training keeps falling, early stopping should trigger.

Step 4.4: Train and save the model

- **What:** run the training loop per SKU (or one shared model with a SKU embedding), then save the best weights.
- **How:** the loop is: forward pass, compute loss, `loss.backward()`, `optimizer.step()`, `optimizer.zero_grad()`. Clip gradients (`clip_grad_norm_(…, 1.0)`) to avoid exploding gradients. Write `torch.save(model.state_dict(), "pytorch_lstm.pt")` on each new best validation loss. Also save the scaler and config.
- **Check:** reloading the file reproduces the same predictions on a fixed batch.

Step 4.5: Define the metrics

- **What:** MAE, MAPE, and a safer percentage metric.
- **Why:** MAPE divides by actual sales, so a day with 0 sales gives infinity. Use WAPE (or sMAPE) alongside it.
- **How:**

```
\mathrm{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i-\hat y_i| \qquad \mathrm{MAPE} =
\frac{100}{n}\sum_{i=1}^{n}\left|\frac{y_i-\hat y_i}{y_i}\right| \qquad
\mathrm{WAPE} = \frac{\sum|y_i-\hat y_i|}{\sum y_i}
```

- **Worked example:** actual = [10, 20, 0], predicted = [12, 18, 3]. Errors are 2, 2, 3, so MAE = 7/3 = 2.33. MAPE is undefined (the third actual is 0). WAPE = 7/30 = 23.3%.
- **Check:** compare your function against `sklearn.metrics.mean_absolute_error`.

Step 4.6: Benchmark everything on the same test split

- **What:** evaluate all 5 models on identical test windows and write one results table.
- **Why:** this is your ablation: the evidence for the claim "the LSTM is better".
- **How:** one row per (SKU, model) with MAE, WAPE and training time, saved to `docs/ablation_benchmark_results.csv`. Report the LSTM's improvement over Seasonal Naive as a percentage.
- **Check:** the LSTM wins on most SKUs. If it does not, say so honestly in the report; that is a legitimate finding.

Step 4.7: Add confidence intervals

- **What:** a lower and upper bound around each forecast.
- **Why:** executives should see uncertainty, not a single falsely precise number.
- **How (option A, MC dropout):** keep dropout active at prediction time, run the model 50 times, take the 5th and 95th percentiles. **Option B (residual quantiles):** on validation data compute residuals (actual − predicted), take their 5th and 95th percentiles per horizon step, and add them to the point forecast.
- **Check:** about 90% of actual test values should fall inside a 90% interval. If only 60% do, the intervals are too narrow.

Step 4.8: Expose `forecast(sku, horizon)`

- **What:** one clean function returning a `ForecastResponse` (Phase 1 schema).
- **Why:** this is the single entry point the pricing engine, simulator and API all call.
- **How:** load the model, scaler and the SKU's latest window once; return dates, mean, lower and upper.
- **Check:** unknown SKU raises a clear error.

Step 4.9: Write the tests

- The model file loads and outputs the right shape.
- Predictions are non-negative (clip at 0).
- The LSTM's test MAE is below Naive's.

- All 5 models run on the same test windows.

Phase 5: Economics, pricing and optimization

Goal: learn how demand reacts to price, find the profit-maximizing price, and compute the financial KPIs. This phase is **open** in your status report (Member 2). Output: `src/economics/elasticity.py`, `optimizer.py`, `financial.py`.

Analogy. Elasticity is a rubber band. Raise the price and demand stretches away from you; how far depends on how stretchy the band is. The optimizer finds the price where stretching further costs you more in lost customers than you gain per sale.

Step 5.1: Prepare the regression data

- **What:** per SKU, build a table of `units`, `price`, day of week, event flag and SNAP flag.
- **Why:** elasticity is estimated from history: on days the price was higher, did fewer units sell?
- **How:** drop days with zero units (the log of 0 is undefined), or use `ln(units + 1)` and note it. Take natural logs of both units and price.
- **Check:** at least ~200 usable days per SKU and a non-trivial spread in `ln(price)`.

Step 5.2: Estimate elasticity with log-log OLS

- **What:** fit this regression per SKU:

```
\ln Q_t = a + e\,\ln P_t + \gamma_1 \text{weekday}_t + \gamma_2
\text{event}_t + \gamma_3 \text{SNAP}_t + \varepsilon_t
```

- **Why log-log:** the coefficient `e` is directly the elasticity: the % change in quantity for a 1% change in price.
- **Worked example (two points):** price goes from 4.00 to 4.40 (+10%) and units from 100 to 82.
  1. Log price change: ln(4.40 / 4.00) = ln(1.10) = 0.0953.
  2. Log quantity change: ln(82 / 100) = −0.1985.
  3. Elasticity ≈ −0.1985 / 0.0953 = **−2.08**. A 1% price rise costs about 2.1% of units. OLS does this over thousands of days at once and averages out the noise.
- **How:** `statsmodels.api.OLS(y, sm.add_constant(X)).fit()`; read `e` from `params`, its standard error from `bse`, p-value from `pvalues`, and R^2 from `rsquared`.

- **Check:** most SKUs give a negative `e`.

Step 5.3: Validate and guard each elasticity

- **What:** accept, clip or replace each estimate.
- **Why:** with only 5 SKUs, some estimates will be noisy or have the wrong sign (promotions that raised both price-tag visibility and sales can fake a positive `e`). Feeding a bad `e` to the optimizer produces absurd prices.
- **How:** flag `e ⩾ 0` or p-value above 0.05. For flagged SKUs fall back to a configurable category prior (e.g. −1.5). Always clip to a sane range such as −3.0 to −0.3. Store `e`, standard error, p-value, R^2 and a `reliable` flag in the response.
- **Check:** no SKU reaches the optimizer with a positive elasticity.

Step 5.4: Build the demand function

- **What:** the iso-elastic demand curve: demand at any price P given a baseline.

```
Q(P) = Q_{base}\left(\frac{P}{P_{base}}\right)^{e}
```

- **Why:** this is how the pricing engine asks "what if I charge P?" without retraining anything.
- **How:** `Q_base` is the LSTM's forecast units per day (Phase 4) at the current price `P_base`. Apply it for each horizon day.
- **Check:** at `P = P_base` the function returns exactly `Q_base`; a higher price returns less demand.

Step 5.5: Set the cost assumption

- **What:** M5 has no cost data, so define unit cost `C = COST_RATIO × P_base` (default 70%).
- **Why:** profit needs a cost. Hiding this assumption would make your results look more factual than they are.
- **How:** keep `COST_RATIO` in `config.py`, allow a per-request override in the API, and state the assumption in the dashboard and report.
- **Check:** changing `COST_RATIO` changes the optimal price in the expected direction (higher cost means higher optimal price).

Step 5.6: Grid-search optimizer

- **What:** try many candidate prices and keep the most profitable one.
- **How:** build a price grid from `0.7·P_base` to `1.3·P_base` in ~200 steps. For each price compute `Q(P)`, then profit `π(P) = (P − C)·Q(P)`. Take the argmax. Add constraints: price at least cost (floor), and a maximum allowed change, e.g. ±30%.
- **Worked example:** base price 4.00, base demand 100/day, e = −2, cost 2.80.
  1. Current profit: (4.00 − 2.80) × 100 = **120.0**.
  2. At P = 5.00: Q = 100 × (1.25)^−2 = 64; profit = 2.20 × 64 = 140.8.
  3. At P = 5.20 (the +30% cap): Q = 100 × (1.30)^−2 = 59.17; profit = 2.40 × 59.17 = **142.0**.
  4. The grid returns 5.20 because it hits the band's edge (see Step 5.7 for why).
- **Check:** the optimum is not on the band edge unless the maths says it should be. If it is, report that it is constrained.

Step 5.7: Cross-check with the closed-form optimum and SciPy

- **What:** verify the grid result two independent ways.
- **Why:** two methods agreeing is how you trust an optimizer.
- **The derivation:** set the derivative of profit to zero:

```
\pi(P)=(P-C)\,Q_{base}\left(\tfrac{P}{P_{base}}\right)^{e} \;\Rightarrow\;
\frac{d\pi}{dP}=0 \;\Rightarrow\; P^{*}=\frac{C\,e}{1+e}\quad (e<-1)
```

- **Worked example (same numbers):** P* = 2.80 × (−2) / (1 − 2) = −5.60 / −1 = **5.60**. Profit there: Q = 100 / 1.4² = 51.02, so π = 2.80 × 51.02 = **142.86**. That is +19% over the current 120.0, with units down 49%. The ±30% cap (5.20) stopped the grid short, which is exactly what Step 5.6 showed.
- **If |e| < 1** there is no finite optimum: raising price always raises profit until it hits the cap. Handle this case explicitly and return the cap with a warning.
- **How:** `scipy.optimize.minimize_scalar(lambda p: -profit(p), bounds=(lo, hi), method="bounded")`.
- **Check:** grid, SciPy and the closed form agree within one grid step when the optimum is inside the band.

Step 5.8: Financial planner

- **What:** convert price and demand into business KPIs over the horizon.

- **How:** for each horizon day compute units `Q`, revenue `R = P·Q`, cost of goods `C·Q`, gross profit `R − C·Q`, and margin `(P − C)/P`. Sum over the horizon. Breakeven units = fixed cost ÷ unit contribution; with an assumed fixed cost of 500 and contribution 2.80 per unit, breakeven = 500 / 2.80 ≈ 179 units.
- **Why:** executives think in revenue, profit and margin, not in elasticities.
- **Check:** revenue minus total cost equals profit exactly; margin lies between 0 and 1.

Step 5.9: Package the output

- **What:** one `PricingResponse` (Phase 1 schema) holding current price, optimal price, expected units, revenue, profit, margin, elasticity details, a `constrained` flag and warnings.
- **Why:** the simulator, API, dashboard and LLM all consume this single object.

Step 5.10: Write the tests

- The grid optimum matches `C·e/(1+e)` when it lies inside the band.
- Profit is concave around the optimum (profit at P*±δ is lower than at P*).
- A positive or insignificant elasticity triggers the fallback.
- `|e| < 1` returns the cap with a warning.
- The price floor is never below cost.

Step 5.11: Merge

- Open a pull request from `feat/pricing-finance` into `main`, make sure CI is green, and notify Member 3 so the simulator can swap its stubs for the real engine.

Phase 6: Counterfactual simulator and API

Goal: answer "what if?" by re-running the same pipeline under a shock, and expose everything over HTTP. Marked done in your report (Member 3, 44 tests). Output: `src/simulation/`, `src/api/`.

Analogy. A flight simulator. You do not crash the real plane; you change one input (engine failure, crosswind) and watch the same aircraft model respond. The key rule: the baseline and the scenario must go through identical code, so any difference comes from the shock and nothing else.

Step 6.1: Define a scenario specification

- **What:** one small object holding every knob a shock can turn: `price_change_pct`, `demand_multiplier`, `cost_multiplier`, `competitor_price_change_pct`, `cross_elasticity`.
- **Why:** all five scenarios become just different values of the same five numbers. That keeps the engine simple and testable.
- **Check:** a scenario with all defaults (0%, ×1, ×1) reproduces the baseline exactly.

Step 6.2: Build one reusable pipeline function

- **What:** `run_pipeline(sku, spec)` does forecast → demand curve → pricing → financials and returns a result object.
- **Why:** one function used for both baseline and scenario guarantees a fair comparison.
- **How:** take the LSTM forecast as `Q_base`, multiply it by `demand_multiplier`, multiply the cost by `cost_multiplier`, apply the price change, then call the Phase 5 pricing and financial functions.
- **Check:** unit-test it with a fixed fake forecast so results are deterministic.

Step 6.3: Implement the five scenarios

Scenario	How it is applied
Price policy change	Set a new price (e.g. +5%); demand reacts via the elasticity
Macro demand shock	`demand_multiplier = 0.85` (−15% demand)
Supply-chain cost inflation	`cost_multiplier = 1.10` (+10% cost)
Competitor price war	Competitor cuts price 20%; our demand multiplier = 1 − cross_elasticity × 0.20, so cross-elasticity 1.0 gives −20% demand
Leeroy & Leeroy stagflation	`cost_multiplier = 1.10` and `demand_multiplier = 0.92` together

- **Worked example** (price held at 4.00, base demand 100/day, cost 2.80, baseline profit = 1.20 × 100 = **120**):
  1. Demand shock: Q = 85, profit = 1.20 × 85 = 102, change **−15.0%**.

2. Cost inflation: cost = 3.08, margin = 0.92, profit = 0.92 × 100 = 92, change **−23.3%**. A 10% cost rise hits profit harder than 10% because the margin per unit is thin.
3. Price war: Q = 80, profit = 1.20 × 80 = 96, change **−20.0%**.
4. Stagflation: margin 0.92 and Q = 92, profit = 84.64, change **−29.5%**. The two shocks compound: you earn less per unit on fewer units.

- **Why it matters:** this reproduces the Leeroy & Leeroy insight that a price rise alone cannot fully offset a demand loss, because the higher price loses more volume through elasticity.
- **Optional:** offer a "react" mode that re-runs the optimizer after the shock to show how much a smart price response recovers.

Step 6.4: Write `simulate_scenario`

- **What:** `simulate_scenario(sku, scenario_name, params)` runs the baseline and the scenario and returns three things: baseline KPIs, scenario KPIs, and deltas (absolute and percent) for units, revenue, profit and margin.
- **Check:** deltas equal scenario minus baseline, to rounding. An unknown scenario name raises a clear error.

Step 6.5: Create the FastAPI app

- **What:** a `FastAPI()` app that loads heavy objects once at startup, not on every request.
- **Why:** loading an LSTM per request would make every call take seconds.
- **How:**

```
from fastapi import FastAPI, HTTPException
from contextlib import asynccontextmanager

state = {}

@asynccontextmanager
async def lifespan(app):
    state["model"] = load_model()      # once, at startup
    state["scaler"] = load_scaler()
    yield

app = FastAPI(lifespan=lifespan)

@app.get("/health")
def health():
    return {"status": "ok"}
```

Step 6.6: Add the endpoints

- `GET /health`: returns ok, used by tests and deployment checks.
- `POST /forecast`: takes a `ForecastRequest`, returns a `ForecastResponse`.
- `POST /pricing/optimize`: takes a `PricingRequest` (SKU, cost ratio, price band), returns a `PricingResponse`.
- `POST /simulate`: takes a `ScenarioRequest`, returns baseline, scenario and deltas.
- Use the Phase 1 Pydantic schemas as `response_model` so FastAPI validates both directions and generates docs at `/docs` for free.

Step 6.7: Handle errors properly

- **What:** return meaningful status codes: 422 for malformed input (automatic), 404 for an unknown SKU, 400 for an out-of-range parameter, 500 only for real bugs.
- **Why:** the dashboard needs to show a helpful message, not a stack trace.

Step 6.8: Write the tests

- Use `fastapi.testclient.TestClient` (needs `httpx`).
- One test per scenario checking the direction of each delta (a cost rise must lower profit).
- The baseline-equals-scenario test from Step 6.1.

- Bad input: negative horizon, unknown SKU, malformed JSON.
- Each endpoint returns the correct status code and matches its schema.
- Aim for broad coverage; your report's 44 tests are a good target.

Step 6.9: Integrate with Phase 5

- Until Member 2 finishes, the simulator can call a stub with a fixed elasticity of −1.5. Once Phase 5 merges, replace the stub and re-run all 44 tests; some expected numbers will legitimately change.

Phase 7: Executive dashboard

Goal: one screen where a non-technical executive picks a product and sees the forecast, the best price, the KPIs and the scenario impacts. Marked done in your report (Member 4, 12 tests). Output: `src/dashboard/` (Plotly Dash) and an optional React UI.

Design rule. An executive reads a dashboard in 30 seconds. Every chart must answer one question, and the headline numbers must be visible without scrolling.

Step 7.1: Sketch the layout first

- **What:** a rough wireframe on paper: a SKU selector at the top, KPI cards in a row, the forecast chart, the price-vs-profit chart, the scenario comparison, and later the narrative panel.
- **Why:** deciding layout first avoids rebuilding callbacks later.

Step 7.2: Create a thin API client

- **What:** a small module with functions `get_forecast`, `get_pricing`, `get_simulation` that call the FastAPI endpoints with `httpx` or `requests`.
- **Why:** the dashboard then never touches model code. It only shows what the API returns, so the same API can later feed the React UI.
- **Check:** each function returns parsed data or raises a clear error on timeout.

Step 7.3: Build the Dash layout

- **What:** components such as `dcc.Dropdown` (SKU), `dcc.Slider` (price change %), `dcc.Dropdown` (scenario), KPI card `html.Div`s, and `dcc.Graph` containers.

- **How:** keep the layout in one function that returns the component tree; keep styling in a CSS file so colours and fonts are easy to change.

Step 7.4: Forecast chart with uncertainty

- **What:** a line chart of the mean forecast with a shaded band between the lower and upper bounds, plus recent actuals for context.
- **How:** in Plotly, add the upper line, then the lower line with `fill="tonexty"` and a translucent colour, then the mean line on top.
- **Why:** the shaded band communicates uncertainty visually.

Step 7.5: Price-versus-profit curve

- **What:** profit on the y-axis against candidate prices on the x-axis, with a marker at the optimal price and a line at the current price.
- **Why:** this is the most persuasive chart in the project. The viewer sees the hill and where you are on it. The gap between your marker and the peak is the opportunity.
- **How:** plot the profit array from the optimizer's grid directly; mark the argmax with a scatter point and a text label.

Step 7.6: KPI cards

- **What:** cards for revenue, profit, margin and units over the horizon, each with the change vs the current price.
- **How:** green for improvement and red for decline, with arrows. Always show units and the time horizon ("next 14 days").

Step 7.7: Scenario comparison chart

- **What:** a grouped bar chart of profit change per scenario (the five scenarios from Phase 6), baseline as the zero line.
- **Why:** it shows at a glance which risks hurt most.

Step 7.8: Wire the callbacks

- **What:** a callback connects each control to the charts it should refresh.
- **How:**

```
@app.callback(
    Output("forecast-graph", "figure"),
    Input("sku-dropdown", "value"),
)
def update_forecast(sku):
    data = get_forecast(sku, horizon=14)
    return build_forecast_figure(data)
```

- Keep figure-building in separate pure functions (`build_forecast_figure`) so they can be unit-tested without running a server.

Step 7.9: Loading, empty and error states

- **What:** show a spinner while the API responds (`dcc.Loading`), an "No data for this SKU" message when empty, and a readable error banner if the API is down.
- **Why:** a blank white chart looks broken; a clear message looks professional.

Step 7.10: Optional React frontend

- **What:** the "Pitch Black & Crimson" React UI with Tailwind, consuming the same endpoints.
- **Why it is optional:** your blueprint classed this as a P2 item. Start it only after Phases 5 and 8 are finished (the risk section of your report flags scope creep). If you build it, enable CORS in FastAPI (`CORSMiddleware`) so the browser can call the API.

Step 7.11: Write the tests

- Each `build_*_figure` function returns a Plotly figure with the expected number of traces.
- Empty or error input does not crash the function.
- Callbacks return the right output types.

Phase 8: LLM narrative and presentation

Goal: turn the numeric results into a short executive summary, safely, and package the project for presentation. This phase is **ready to start** in your status report (Member 5). Output: `src/llm/prompts.py`, `fallback.py`, `narrator.py`, a 13-slide deck, a 5-minute demo script.

Analogy. The LLM is a speechwriter, not an analyst. The analysis is already done by deterministic code. The speechwriter may only rephrase the numbers you hand over; if it invents one, the speech is thrown out and a template speech is read instead.

Step 8.1: Define the structured input

- **What:** one dictionary (or Pydantic object) with every fact the narrative may use: SKU, horizon, forecast total units, current price, optimal price, profit change %, elasticity, constrained flag, and per-scenario profit deltas.
- **Why:** the LLM sees only these facts. Anything not in the dictionary cannot legitimately appear in the output, which makes validation possible.

Step 8.2: Write the prompt template (`prompts.py`)

- **What:** a fixed template with the facts inserted and strict rules.
- **How:** for example:

```
You are a business analyst writing for a non-technical executive.
Use ONLY the numbers in the DATA block below. Do not invent, estimate
or round any number differently. If something is not in DATA, do not
mention it. Write 3 short paragraphs: (1) demand outlook, (2) pricing
recommendation and expected profit impact, (3) main risks from the
scenarios. No jargon. Do not give guarantees.

DATA:
{data_json}
```

- **Why:** constraining the model and putting data in a labelled block cuts hallucination sharply, but does not eliminate it. That is why Steps 8.3 and 8.5 exist.

Step 8.3: Build the deterministic fallback (`fallback.py`)

- **What:** a rule-based generator using f-strings and thresholds. No AI, no network.
- **Why:** it is the offline path and the safety net. It always works and never lies.
- **How:**

```
def fallback_narrative(d):
    direction = "increase" if d["profit_delta_pct"] > 5 else (
                "decrease" if d["profit_delta_pct"] < -5 else "hold roughly
steady")
    return (
        f"Demand for {d['sku']} is forecast at {d['units']:,.0f} units over "
        f"the next {d['horizon']} days. Moving the price from "
        f"{d['price']:.2f} to {d['opt_price']:.2f} is expected to make profit
"
        f"{direction} by {abs(d['profit_delta_pct']):.1f}%."
    )
```

- Add rules for the worst scenario ("the largest downside is X, cutting profit by Y%") and for the `constrained` flag ("the optimum is limited by the allowed price range").
- **Check:** unit-test several inputs including negative, zero and borderline (±5%) profit changes.

Step 8.4: Build the API path (`narrator.py`)

- **What:** a function that calls the LLM (GPT-3.5, Llama via a hosted API, or Claude) with the template.
- **How:** read the API key from an environment variable (never commit it), set a timeout (e.g. 15 seconds), temperature low (0.2) for consistency, and cap output tokens.
- **Check:** with a missing key or a forced timeout, the function does not crash.

Step 8.5: Validate the output numerically

- **What:** before showing LLM text, check that it contains no numbers that are not in the input.
- **How:** extract all numbers with a regex such as `-?\d[\d,]*\.?\d*`, normalise (strip commas, allow rounding to 0, 1 or 2 decimals), and require each to match a value in the input data. Allow harmless numbers such as paragraph labels or the horizon.
- **Why:** this is the actual hallucination guard. It turns "trust the model" into "verify the model".
- **Check:** feed a deliberately wrong sentence ("profit rises 80%") and confirm it is rejected.

Step 8.6: Orchestrate with fallback

- **What:** the final `generate_narrative(data)` tries the API, validates, and on any failure (timeout, error, empty text, failed validation) returns the fallback text.
- **Also:** return a `source` field (`"llm"` or `"fallback"`) so the dashboard can show which one was used.

Step 8.7: Always append the disclaimer

- **What:** add "AI-assisted, not binding." to every narrative, from either path.
- **Why:** it addresses the user-trust risk named in both of your source documents.

Step 8.8: Add the endpoint and dashboard panel

- **What:** `POST /narrative` takes the structured data (or a SKU and scenario) and returns text and source. In the dashboard add a "Executive summary" card with a "Generate" button.
- **Check:** the endpoint returns the fallback when no API key is configured.

Step 8.9: Build the 13-slide pitch deck

1. Problem: executives lack one tool for forecast, price and risk.
2. Solution: NAOMI in one sentence and one picture.
3. Architecture: the pipeline from the Overview.
4. Data: M5, 5 SKUs, 1,913 days, why it is credible.
5. Forecast results: the ablation table, LSTM vs baselines.
6. Elasticity: what it means, the estimates, the reliability flags.
7. Optimizer: the price-vs-profit curve and the opportunity.
8. Scenarios: the five "what if" results.
9. Dashboard: screenshots or a live view.
10. LLM layer: the narrative and the fallback safety design.
11. Results vs blueprint: what you planned and what you exceeded.
12. Roadmap and limitations: honest list (cost assumption, 5 SKUs).
13. Ask or next steps.

Step 8.10: Write the 5-minute demo script

1. 0:00-0:30: the problem, in plain words.

2. 0:30-1:15: the data and the forecast chart.
3. 1:15-2:15: pick a SKU, show the price-vs-profit curve and the recommendation.
4. 2:15-3:15: run two scenarios live (cost inflation, price war).
5. 3:15-4:00: generate the narrative, mention the fallback and the disclaimer.
6. 4:00-4:30: results vs the blueprint.
7. 4:30-5:00: limitations and next steps.

Rehearse with a timer and prepare a recorded backup in case the live demo fails.

Step 8.11: Write the tests

- The fallback produces non-empty text for every input.
- The validator accepts correct numbers and rejects invented ones.
- The orchestrator falls back on timeout and on validation failure.
- The disclaimer is always present.